Ledger Calculus — Program sebagai Buku Besar Resource

Oct 30, 2026 · @Ricky Raihan Azhari

Ringkasan

Ledger Calculus adalah program riset, bukan teori yang sudah mapan. Gagasannya: setiap program diperlakukan sebagai entri buku besar. Nilai, ownership, protocol, effect, dan biaya dicatat dalam satu derivasi, dan setiap komposisi harus "balance".

Nama ini merangkum aksioma moralnya: **tidak ada resource tanpa asal, tidak ada effect tanpa catatan, tidak ada biaya di luar pembukuan.** Core calculus-nya disebut **λ-Ledger**.

Kontribusi yang dituju bukan potongan teori baru. Potongannya sudah ada di ekosistem Iris dan resource analysis. Targetnya adalah *menyatukan* potongan itu ke satu judgment yang bisa di-infer untuk **Zig**, bahasa dengan alokasi dan I/O eksplisit. ZBeam berperan sebagai pengguna pertama dan bukti kelayakan.

1. Problem

Saat ini setiap pertanyaan tentang program dijawab oleh tool yang berbeda, padahal jawabannya saling bergantung.

Tool	Pertanyaan yang dijawab
Type checker	Apakah tipe benar?
Effect system	Apakah program melakukan I/O, state, atau throw?
Borrow checker	Siapa yang memiliki resource?
Verifier	Apakah algoritmanya benar?
Race detector / model checker	Apakah concurrency aman?
Profiler	Berapa mahal eksekusinya?

Contoh di mana semua lapisan bertabrakan:

1. Task memulai async read dan menyerahkan buffer.

2. Task dibatalkan.
3. Cleanup membebaskan buffer.
4. Kernel masih menulis ke buffer itu, lalu completion datang terlambat.

Ini sekaligus masalah correctness, ownership, concurrency, cancellation effect, temporal memory safety, dan biaya menahan resource. Tidak satu pun tool di tabel bisa menangkapnya sendirian. Ledger Calculus ingin memberi **satu specification** untuk seluruhnya.

2. Posisi terhadap riset yang ada

Sebagian besar komponen Ledger sudah diselesaikan secara terpisah, dan ekosistem Iris sudah menutup tiga dari lima. Celah yang tersisa sempit tetapi nyata.

Komponen	Karya yang ada	Sudah tercakup	Belum tercakup
Concurrent state & ownership	Iris (Jung dkk., ESOP 2017; JFP 2018)	Separation logic higher-order, invariant, ghost state	Bukti manual interaktif di Rocq
Biaya di separation logic	Time credits & time receipts in Iris (Mével, Jourdan, Pottier, ESOP 2019)	Upper bound waktu sebagai resource logis	Satu dimensi (waktu), tetap manual
Effect handlers	Hazel (de Vilhena & Pottier, POPL 2021)	Reasoning handler di Iris	Hanya continuation one-shot, tanpa biaya
Message-passing protocol	Actris (Hinrichsen, Bengtson, Krebbers, POPL 2020)	Dependent separation protocols + functional correctness	Tanpa biaya
Batas resource otomatis	AARA / RaML (Hoffmann dkk.)	Inference bound polinomial lewat LP	Effects dan concurrent state terbatas
Protocol + biaya	Resource-aware session types (Das, Hoffmann, Pfenning, LICS 2018); Nomos (CSF 2021)	Work dan span untuk proses message-passing	Tanpa shared memory dan handler

Komponen	Karya yang ada	Sudah tercakup	Belum tercakup
Grade kuantitatif	QTT (Atkey, LICS 2018), Idris 2, Granule (ICFP 2019)	Usage dan grade di sistem tipe	Tanpa program logic untuk concurrency

Celahnya ada tiga:

1. **Belum ada satu judgment** yang memuat ownership, protocol, effect, correctness, dan biaya sekaligus.
2. **Biaya multi-dimensi di bawah concurrency** (terutama peak memory) belum punya aturan komposisi yang sound dan scheduler-independent.
3. **Otomasi vs ekspresivitas**: Iris ekspresif tetapi manual, sedangkan AARA otomatis tetapi sempit. Belum ada sistem bertingkat yang menghubungkan keduanya dalam satu model semantik.

Konsekuensinya, strategi paling realistis adalah **membangun di atas Iris** (memakai time credits, Hazel, dan Actris sebagai fondasi), bukan membuat logic baru dari nol.

3. Bentuk judgment

Satu entri ledger mencatat apa yang masuk, apa yang keluar, dan berapa harganya:

```
\Gamma;\ \Delta \vdash e : A \dashv \Delta' \quad \{P\} \rightsquigarrow \
{Q\} \quad !\,\varepsilon \quad @\,\kappa
```

- Γ: konteks biasa, boleh dipakai berulang.
- Δ dan Δ′: resource linear sebelum dan sesudah eksekusi.
- A: tipe hasil.
- P dan Q: precondition dan postcondition logis.
- ε: effect yang mungkin terjadi, sebagai elemen effect quantale (urutan bermakna).
- κ: vektor biaya upper bound.

Contoh untuk async read berbasis completion queue:

```
read(fd, buf, n) : Result(usize)

Precondition:   fd terbuka; buf dimiliki task; len(buf) ≥ n
Ownership:      buf → InFlight(token)          (bukan lagi milik task)
Effects:        KernelIO · MaySuspend · MayCancel
Cost:           W ≤ a·n + b
                SQE ≤ 1, CQE ≤ 2              (completion + cancel ack)
                M_retained ≤ n + c
Postcondition:  token diselesaikan → buf kembali ke task
```

Biaya dihitung dalam satuan entri submission/completion queue, bukan jumlah syscall, karena submission bisa di-batch. Tipe `Result(usize)` saja tidak mengatakan siapa memiliki buffer, apakah operasi bisa dibatalkan, atau kapan buffer aman dibebaskan.

4. Hipotesis sentral

> **Ledger Hypothesis.** Untuk subset bahasa sistem dengan effect eksplisit, structured concurrency, dan resource linear, ada satu model semantik tempat empat tingkat pengecekan sound sekaligus, dan spesifikasi komponen dapat dikomposisi tanpa membuka implementasinya.

Versi sebelumnya mengklaim seluruh sistem decidable. Itu tidak mungkin bila functional correctness ikut diperiksa (Rice, 1953). Maka decidability dibagi per tingkat:

Tingkat	Yang diperiksa	Status	Beban programmer
0	Tipe, ownership linear, protocol state, effect row	Decidable, otomatis	Tidak ada
1	Batas biaya κ	Sound, tidak lengkap; inference via LP ala AARA	Anotasi untuk rekursi non-struktural
2	Functional correctness P ⇒ Q	Verification condition ke SMT; semi-decidable	Loop invariant, lemma
3	Properti di luar jangkauan otomasi	Bukti interaktif di Rocq di atas model Iris	Bukti manual

Yang dijanjikan bukan "sistem selalu bisa membuktikan semuanya". Yang dijanjikan: **apa pun yang lolos di tingkat mana pun benar-benar valid terhadap model yang sama**, dan tingkat yang lebih tinggi bisa dipakai hanya untuk bagian kode yang membutuhkannya.

5. Struktur aljabar

Ledger memakai empat struktur. Masing-masing sudah punya teori matang, dan yang baru hanyalah cara keempatnya disatukan.

Struktur	Mencatat	Bentuk	Fondasi
Refinement / dependent types	Correctness nilai	sort(xs) : { ys | sorted(ys) ∧ perm(xs, ys) }	Liquid types, Iris
Separation algebra	Ownership	R ∗ S; tidak ada R ⊢ R ∗ R	Iris, QTT
Effect quantale	Effect berurutan	Read(fd) · Write(log) · MayCancel	Gordon (ECOOP 2017), algebraic effects
Cost vector	Biaya	κ = (W, S, M, A, N)	AARA, time credits

Komponen biaya: W total work, S span (critical path), M peak memory, A jumlah alokasi, N jumlah pesan atau entri queue.

Aturan komposisi biaya

W dan S mengikuti aturan standar:

```
W(P;Q) = W(P) + W(Q) \qquad S(P;Q) = S(P) + S(Q)
```

```
W(P \parallel Q) = W(P) + W(Q) \qquad S(P \parallel Q) \le \max(S(P), S(Q)) + c_{\text{sync}}
```

Peak memory dulu dibiarkan terbuka. Perbaikannya: catat juga R, memori yang masih ditahan setelah program selesai. Dengan itu ada aturan yang sound untuk scheduler mana pun:

```
M(P;Q) \le \max\bigl(M(P),\ R(P) + M(Q)\bigr) \qquad M(P \parallel Q) \le M(P) + M(Q)
```

Aturan parallel itu pesimis, tetapi tidak bergantung pada scheduler. Bound yang lebih ketat hanya diizinkan bila jadwalnya dibatasi, misalnya $\kappa(e, \sigma)$ untuk kelas scheduler σ tertentu.

6. Hukum dasar

Ini bukan "aksioma alam". Ini adalah hukum yang harus dipenuhi struktur aljabarnya, lalu dibuktikan sebagai metatheorem. Kolom terakhir mencatat bagian yang diperbaiki dari versi sebelumnya.

#	Hukum	Rumus	Catatan
1	Identity	$\mathrm{skip} ; g = g ; \mathrm{skip} = g$	Biaya skip = 0
2	Associativity	$(g_1 ; g_2) ; g_3 = g_1 ; (g_2 ; g_3)$	Compiler bebas mengelompokkan
3	Resource conservation	$\Delta = \Delta_1 \uplus \Delta_2$ untuk $P \parallel Q$	Resource dibagi, atau dibagikan lewat invariant yang dibuktikan
4	Frame	$\{P\}\ e\ \{Q\} \Longrightarrow \{P * R\}\ e\ \{Q * R\}$	Inti modularitas separation logic
5	Effect ordering	$\mathrm{Lock} \cdot \mathrm{Write} \neq \mathrm{Write} \cdot \mathrm{Lock}$	Commute hanya dengan bukti noninterference
6	Choice bound	$\varepsilon(\mathrm{if}) = \varepsilon_b \cdot (\varepsilon_1 \sqcup \varepsilon_2)$; $\kappa(\mathrm{if}) \leqslant \kappa_b + \max(\kappa_1, \kappa_2)$	**Diperbaiki:** biaya dan effect evaluasi kondisi b ikut dihitung
7	Parallel noninterference	$\mathrm{res}(P) \cap \mathrm{res}(Q) = \varnothing$, atau akses bersama di bawah invariant	$\parallel$ komutatif hanya sampai observational equivalence
8	Handler homomorphism	$H(e_1 ; e_2) = H(e_1) ; H(e_2)$; $\kappa(H(e)) \leqslant \Phi_H(\varepsilon(e), \kappa(e))$	**Diperbaiki:** hanya untuk deep handler dengan continuation one-shot (linear)
9	Monotonic refinement	$P \Rightarrow P'$, $Q' \Rightarrow Q$, $\varepsilon' \preceq \varepsilon$, $\kappa' \preceq \kappa$	Implementasi boleh menjanjikan lebih kuat

Pembatasan di hukum 8 disengaja. Continuation multi-shot bisa menggandakan resource linear dan melipatgandakan biaya. Mengikuti Hazel, λ-Ledger memperlakukan continuation sebagai resource linear. Multi-shot hanya boleh bila continuation tidak

menangkap resource linear apa pun.

7. Metatheorem

Syntax dan typing rules belum cukup. Kalkulus ini baru bernilai bila hasil berikut terbukti dan dimekanisasi.

1. **Adequacy.** Derivasi di logic menjamin perilaku di operational semantics. Ini teorema induk, dan yang lain diturunkan darinya, seperti di Iris.
2. **Preservation.** Jika $e \to e'$, maka $\mathrm{Spec}(e') \preceq \mathrm{Spec}(e)$.
3. **Progress.** Program tidak stuck, kecuali pada blocking atau suspension yang dinyatakan di ε.
4. **Ownership safety.** Resource linear tidak dipakai setelah dipindahkan atau dibebaskan.
5. **Protocol fidelity.** Pesan dan operasi mengikuti state protocol.
6. **Race freedom.** Tidak ada akses concurrent ilegal ke resource yang sama.
7. **Cost soundness.** Jika diturunkan $\kappa(e) \leqslant f(n)$, maka setiap eksekusi dalam model memenuhi batas itu.
8. **Contextual refinement.** Tidak ada konteks valid yang bisa mengamati pelanggaran spesifikasi.
9. **Compiler preservation.** Kompilasi mempertahankan safety, effect yang observable, protocol, dan relasi biaya $\mathrm{cost_machine} \leqslant a \cdot \mathrm{cost_source} + b$.

8. Target: Zig, dengan ZBeam sebagai bukti kelayakan

Ledger paling berdampak bila dipasang di level bahasa, bukan di satu program. Targetnya Zig: satu kontrak di `Allocator` dan `std.Io` berlaku untuk setiap program yang memakainya.

Kenapa Zig

Zig tidak punya control flow atau alokasi tersembunyi. Karena itu, sebagian besar entri ledger sudah terlihat langsung di kode.

Fitur Zig	Komponen ledger	Artinya
`Allocator` sebagai parameter eksplisit	Biaya A, M + ownership	Setiap alokasi punya asal yang jelas

Fitur Zig	Komponen ledger	Artinya
`std.Io` sebagai parameter eksplisit	Effect handler	Backend threaded dan evented adalah handler; hukum 8 berlaku: perilaku sama, biaya beda
Error union + `try`	Effect exception	Jalur error terlihat di tipe
`defer` / `errdefer`	Urutan effect dan cleanup	Bisa dibuktikan setiap alokasi dibebaskan di semua jalur error
`comptime`	Spesifikasi sebagai tipe	Wrapper seperti `Owned(T)` dan `InFlight(T)` bisa dideklarasikan tanpa mengubah bahasa

Celah: ownership tidak ada di sistem tipe

Zig tidak punya ownership maupun linearitas. Pointer bebas di-alias, nilai bisa disalin, dan safety check hanya berjalan saat runtime di Debug/ReleaseSafe. `comptime` juga tidak bisa membaca isi body fungsi, jadi checker tidak bisa dibuat sebagai library biasa.

Solusinya adalah **checker eksternal yang membaca ZIR/AIR**, dengan anotasi berupa wrapper type `comptime`. Checker hanya berlaku untuk subset Zig: tanpa `@ptrCast` bebas, tanpa inline asm, dan tanpa `@intToPtr` di kode yang diverifikasi. Binary tetap dikompilasi oleh compiler Zig biasa.

Kontrak inti

1. **Allocator.** Memori hanya boleh dibebaskan oleh allocator yang mengalokasikannya. `arena.reset()` membatalkan semua pointer turunannya.
2. **Future dari `std.Io`.** Setiap future harus di-await atau di-cancel tepat satu kali (resource linear). Buffer yang diserahkan baru kembali setelah future selesai.
3. **Leak di jalur error.** `errdefer` ditambah aturan ownership membuktikan tidak ada kebocoran walaupun error muncul di tengah fungsi. Sekarang hal ini hanya dideteksi saat runtime oleh `std.testing.allocator`.
4. **Kontrak biaya std.** Contohnya `ArrayList.append`: work amortized O(1), alokasi ⩽ 1. Bound program pengguna disusun dari kontrak ini.

Posisi

Rust sudah punya fondasi formal (RustBelt, di atas Iris) dan beberapa verifier yang matang (Verus, Prusti, Creusot). Zig belum punya padanan RustBelt, tetapi bukan berarti kosong:

- CLR (Checker for Lifetimes and other Refinement types) menganalisis AIR untuk membawa jaminan memory safety setara Rust ke Zig. Ia memakai fork compiler Zig yang meneruskan AIR ke plugin eksternal.
- Zirakzigil menambahkan contracts, refinement types, ownership backstop, dan SMT verifier di atas compiler Zig standar, tanpa syntax baru.
- zig-spec adalah upaya resmi untuk spesifikasi Zig yang dimekanisasi, tetapi baru dimulai.

Artinya, pendekatan "checker eksternal di atas AIR" sudah punya prior art. Kontribusi Ledger yang tersisa ada tiga: (1) biaya, effect, dan protocol, bukan hanya lifetime; (2) kontrak `std.Io` dan future-nya; (3) soundness yang dibuktikan secara foundational di Iris. CLR adalah kandidat basis untuk fase 4, bukan pesaing.

Bukti kelayakan: ZBeam

ZBeam dipakai sebagai pengguna pertama dan bukti kelayakan karena invariant-nya sudah eksplisit di desain, meskipun belum formal. ZBeam adalah Erlang Distribution peer native di Zig 0.17, tanpa tracing GC, dengan ownership eksplisit. Statusnya MVP: implementasi I/O masih blocking TCP framing, dan `io_uring` beserta arena ownership masih ditandai "research only".

README ZBeam juga mencatat bahwa batas proses OS tidak mengisolasi actor satu sama lain di dalam ZBeam. Artinya, disiplin ownership adalah *satu-satunya* isolasi. Karena itu Ledger relevan di sini.

Invariant yang sudah ada dan bentuk ledger-nya

Invariant ZBeam (sekarang)	Komponen ledger	Bentuk spesifikasi
Mailbox bounded	Biaya M, A	M_mailbox ⩽ k · frame_max untuk kapasitas k
Demand-gated read: demand dipesan sebelum read, tanpa prefetch frame	Protocol + ownership	read : Demand ⊸ Frame; satu token demand untuk satu frame

Invariant ZBeam (sekarang)	Komponen ledger	Bentuk spesifikasi
Decode ETF menghasilkan owned copy, dengan batas alokasi agregat	Ownership + biaya A	Term hasil decode tidak meminjam input buffer; A ⩽ limit
Owner M2c: satu call per worker, uncertain work tidak pernah di-replay	Protocol at-most-once	Admitted → Sent → (Replied | Uncertain); Uncertain terminal, tanpa transisi Retry
Tabel koneksi tetap 64 slot	Resource conservation	64 token slot di Δ; tidak ada slot tanpa asal
Stop/join kooperatif; handler CPU-bound tidak di-preempt	Effect + biaya W	Lihat di bawah

Baris terakhir memberi wawasan konkret. Karena cancellation bersifat kooperatif, **bound W pada handler sekaligus menjadi bound latency stop**. Tanpa κ, spesifikasi hanya bisa bilang "stop akan terjadi pada titik yield berikutnya". Dengan κ, ia bisa bilang "stop terjadi dalam ⩽ W_handler langkah".

Protocol `io_uring` + arena ownership

Di bagian inilah Ledger paling berguna, karena bug cancellation di `io_uring` tidak tertangkap tipe biasa. Protocol buffer per operasi:

```
Owned ──submit──▶ InFlight ──CQE(result)──▶ Completed
                    │
                  cancel
                    ▼
              CancelRequested ──CQE(-ECANCELED)──▶ Cancelled
                    │
                    └──CQE(result) datang duluan──▶ Completed
```

Aturan yang harus dibuktikan:

1. `free(buf)` hanya legal di `Completed` atau `Cancelled`, bukan saat `cancel()` dipanggil.
2. Dari `CancelRequested`, operasi masih bisa selesai *sukses*. Kedua cabang harus ditangani.

3. **Reset arena hanya legal bila tidak ada token `InFlight` yang berasal dari arena itu.** Ini aturan baru yang muncul karena ZBeam berencana memakai arena: satu `reset()` bisa membebaskan banyak buffer sekaligus.
4. Biaya per operasi: SQE ⩽ 1, CQE ⩽ 2, M_retained ⩽ n + c sampai token diselesaikan.

Dari satu derivasi, ZBeam mendapat jaminan tidak ada use-after-free, cancellation tidak membocorkan ownership, completion yang terlambat tetap aman, dan memori yang ditahan punya batas.

Protocol buffer ini bukan khusus ZBeam. Ia adalah instans kontrak 2 (future `std.Io`) untuk backend `io_uring`, jadi bukti yang sama bisa dipakai ulang oleh program Zig lain.

9. Hambatan tersulit

Enam hambatan ini menentukan apakah Ledger bisa dipakai, bukan hanya dibuktikan.

Hambatan	Mengapa sulit	Arah solusi
Interaksi effect	state + exception ≠ exception + state; maknanya bergantung pada urutan handler	Effect quantale non-komutatif; urutan handler masuk ke ε
Continuation multi-shot	Menggandakan resource linear dan biaya	Continuation linear secara default (hukum 8)
Biaya bergantung scheduler	Peak memory, latency, dan contention berubah per jadwal	Bound pesimis yang scheduler-independent; κ(e, σ) bila jadwal dibatasi
Correctness vs performance	Lock dan copy menambah safety sekaligus biaya	κ membuat harga keamanan terlihat, bukan disembunyikan
Abstraksi menyembunyikan biaya	Interface modular menutupi implementasi	Cost contract: interface membuka fungsi biaya simbolik, bukan kode
Bahasa target tanpa anotasi	Zig tidak punya ownership atau linearitas di tipe, dan comptime tidak bisa membaca body fungsi	Checker eksternal di atas ZIR/AIR, anotasi sebagai wrapper type comptime, untuk subset tanpa @ptrCast bebas dan inline asm

Ada satu risiko khusus target Zig: bahasanya belum 1.0, dan ZIR/AIR berubah tiap rilis. Checker harus dikunci ke satu versi Zig per rilis, dan kontrak `Allocator` / `std.Io` ditulis terhadap antarmukanya, bukan terhadap detail internalnya.

10. Kriteria keberhasilan dan roadmap

Ledger dianggap berhasil bila ada core calculus, semantics, soundness proof yang dimekanisasi, checker otomatis tingkat 0–1 untuk subset Zig, dan ZBeam lolos checker itu. Setiap fase punya gate yang terukur.

1. **Fase 1 — λ-Ledger sekuensial.** Ownership, effect, dan biaya (W, A) dimekanisasi di Rocq di atas Iris dan time credits.
   - Gate: teorema adequacy terbukti.
2. **Fase 2 — protocol, concurrency, dan model ZBeam.** Ditambah protocol ala Actris dan biaya S, M, N.
   - Gate: model demand-gated mailbox dan protocol at-most-once M2c ZBeam terverifikasi.
3. **Fase 3 — kontrak `Allocator` dan `std.Io`.** Kontrak ditulis dan dibuktikan untuk satu allocator (arena) dan dua backend Io.
   - Gate: kedua backend Io terbukti setara perilakunya dan hanya berbeda biaya.
4. **Fase 4 — checker Zig di atas ZIR/AIR.** Inference tingkat 0–1 untuk subset Zig.
   - Gate: modul mailbox/dispatcher ZBeam lolos tanpa bukti manual, dan leak di jalur `errdefer` pada test suite buatan tertangkap semua.
5. **Fase 5 — `io_uring` + arena di ZBeam.** Protocol buffer dan aturan reset arena diterapkan.
   - Gate: cancellation-safe read terbukti, dan test SIGKILL/nodedown ZBeam yang sudah ada tetap lolos.

Bentuk akhir yang dituju bagi programmer:

```
// Sketsa API, belum ada. Spesifikasi menempel di samping fungsi;
// signature dan body readFrame tidak berubah.
pub const readFrame_spec = ledger.spec(readFrame, .{
    .ownership = .{ .buf = .moved_then_returned },
    .effects = .{ .kernel_io, .may_cancel },
    .work = ledger.linear("buf.len"),
    .alloc = 0,
    .cancel = .safe,
});
```

Checker lalu menghasilkan proof certificate, effect manifest, dan resource bound. Binary tetap dibuat oleh compiler Zig biasa.

11. Arah fundamental: cancellable linear futures

Agar setara Curry–Howard, Ledger harus dimulai dari logika, bukan dari sistem spesifikasi. Kandidat intinya adalah **cancellable linear futures**: future linear yang bisa di-cancel tanpa melepaskan resource-nya.

Syarat setara Curry–Howard

1. Logikanya berdiri sendiri: sequent calculus dengan aturan yang bisa dibenarkan tanpa menyebut program.
2. Cut elimination sama dengan eksekusi.
3. Properti penting (bebas deadlock, bebas use-after-free, batas resource) menjadi korolari cut elimination, bukan dibuktikan terpisah.
4. Ada model kategorinya.

Bagian 3–10 belum memenuhi keempatnya. Bagian ini adalah jalur untuk mencapainya.

Prior art

Gagasan	Karya	Status
Linear futures dari Curry–Howard adjoint logic	Back to Futures (Pruiksma & Pfenning, JFP 2022)	Ada; `A ⊸ ◇A` pada dasarnya linear future
◇ sebagai "eventually"	Fair Reactive Programming (Cave dkk.); Lively RaTT (Bahr dkk., POPL 2021)	Ada, untuk nilai yang berubah terhadap waktu, bukan resource

Gagasan	Karya	Status
Resource safety + exception secara Curry–Howard	Curry-Howard for destructors (Congard, Munch-Maccagnoni, Douence, 2025)	Ada; borrowing disebut sebagai masalah terbuka
Semantik borrowing	From Linearity to Borrowing (OOPSLA 2025)	Ada, untuk lifetime leksikal, tanpa pihak luar asinkron
Future linear yang bisa di-cancel tanpa melepas resource	—	Tidak ditemukan

Di praktik, masalahnya nyata dan belum diselesaikan di level logika. uringscope (2026) menyebut pengecekan kontrak buffer-ownership `io_uring` sebagai titik yang belum diisi. withoutboats menyimpulkan kernel harus memiliki buffer. rustfs-uring menegakkan aturan yang sama lewat tabel orphan dan acceptance test, bukan lewat logika.

Bacaan logis

```
submit : A ⊸ ◇ₖA        menyerahkan resource, menerima janji kembali dalam ⩽
k langkah
await  : ◇ₖA ⊸ A         satu-satunya jalan mendapatkan resource lagi
cancel : ◇ₖA ⊸ ◇ₖA       cancel adalah aksi protocol, bukan destruksi
free   : A ⊸ 1           free hanya menerima A
```

Logika ini tidak punya aturan ◇A ⊢ A selain lewat await. Akibatnya, use-after-free saat cancellation bukan bug yang ditangkap checker. Ia adalah bukti yang tidak bisa diturunkan.

Konjektur

> Ada adjoint logic bergraded dengan modality pinjaman $\Diamond_k$ dan aturan cancel di atas, sehingga (1) cut elimination berlaku dan berkorespondensi dengan eksekusi I/O asinkron, (2) panjang normalisasi dibatasi grade, dan (3) bebas deadlock, bebas use-after-free, serta batas memori yang ditahan adalah korolari cut elimination.

Basis dan langkah

Basisnya adjoint logic dari Pruiksma & Pfenning. Logika itu sudah punya Curry–Howard, concurrency, dan linear futures, dan grade biaya datang dari keluarga riset yang sama (Rast).

1. **Logika.** Tambahkan aturan cancel pada future linear dan buktikan cut elimination.
2. **Grade.** Tambahkan $\Diamond_k$ dengan biaya ala Rast, lalu buktikan batas panjang normalisasi.
3. **Term assignment.** λ-Ledger diturunkan dari bukti, bukan dirancang terpisah.
4. **Model kategori.** Cari struktur yang menginterpretasikan $\Diamond_k$ dan cancel.
5. **Turun ke praktik.** Kontrak `std.Io` (bagian 8) dan protocol buffer ZBeam menjadi instans term language, bukan aksioma.

Kalau berhasil, hasilnya sekaligus menjawab sebagian masalah terbuka Congard dkk. untuk kasus borrowing asinkron.

Risiko

- Isi lengkap *Back to Futures* belum dibaca. Klaim bahwa paper itu tidak memodelkan cancellation masih dugaan.
- Literatur cancellation di session types belum ditinjau ulang. Bisa jadi sebagian aturan cancel sudah ada di sana.
- Ini riset logika murni. Cut elimination dengan aturan cancel bisa gagal berlaku, atau terlalu lemah untuk menangkap cancellation I/O nyata.

12. Peta posisi: Fil-C, Zig, dan ZBeam

Ketiganya berada di sumbu yang berbeda. Fil-C mengamankan kode lewat runtime, Zig adalah tempat kontrak tinggal, dan ZBeam adalah instans pertama. Ledger menargetkan satu-satunya kuadran yang, dari pencarian sejauh ini, belum terisi: bukti statis untuk protocol dan biaya.

Kuadran kanan atas masih kosong: bukti statis untuk protocol dan biaya



peta posisi · cara menjamin × cakupan jaminan

Sumbu horizontal adalah cara menjamin, sumbu vertikal adalah cakupan jaminan. Panah putus-putus menunjukkan perpindahan Zig dan ZBeam bila konjektur bagian 11 terbukti. Fil-C tetap di tempatnya dan menjadi batas kepercayaan untuk kode C.

Apa yang terbukti, dan lewat apa

	Jaminan sekarang	Bila konjektur terbukti	Dibuktikan lewat	Tetap tidak terbukti
Fil-C	Memory safety penuh untuk C/C++ yang tidak dimodifikasi; bounds check, InvisiCaps, dan GC; pelanggaran menjadi panic; overhead sekitar 4x di kasus terburuk	Fil-C sendiri tidak dibuktikan Ledger. Kode C yang dikompilasi Fil-C menjadi komponen dengan kontrak memori yang dijamin runtime	Asumsi (aksioma) di batas FFI: kontrak memori dipercaya dari Fil-C, kontrak protocol di batas itu ditulis sebagai asumsi	Protocol dan biaya kode C; kebenaran runtime dan GC Fil-C
Zig (bahasa)	Safety check runtime di Debug/ReleaseSafe; tidak ada ownership di tipe; leak dideteksi `std.testing.allocator` saat test	Kontrak `Allocator` dan `std.Io`: free hanya dari A; future di-await atau di-cancel tepat sekali; buffer kembali hanya lewat await; backend Io setara perilakunya; batas biaya	Kontrak diturunkan sebagai instans term language (bagian 11, langkah 5); kode pengguna diperiksa checker atas AIR (bagian 8, fase 4); soundness checker dari adequacy (bagian 7)	Kode di luar subset (`@ptrCast` bebas, inline asm); logika bisnis

	Jaminan sekarang	Bila konjektur terbukti	Dibuktikan lewat	Tetap tidak terbukti
ZBeam	Invariant desain (mailbox bounded, demand-gated read, at-most-once M2c, tabel 64 slot) dijaga lewat kode dan test SIGKILL/nodedown; `io_uring` belum ada	Tidak ada prefetch frame; Uncertain terminal di M2c; tidak ada use-after-free saat cancel; reset arena hanya tanpa token InFlight; latency stop ⩽ W handler	Protocol mailbox dan M2c diverifikasi sebagai model (fase 2); keamanan cancel adalah korolari cut elimination, karena $\Diamond_k A$ tidak bisa menjadi A tanpa await; batas stop dari grade biaya	Perilaku peer Erlang remote; keamanan cookie dan ketiadaan TLS (dicatat di README); handler CPU-bound tanpa bound W

Fil-C dan Ledger tidak bersaing. Usulan Fil-C ABI di Zig membuka kemungkinan dua lapis: kode C yang di-link dijaga Fil-C saat runtime, sedangkan kode Zig yang dianotasi dibuktikan Ledger saat kompilasi.

13. Rumusan akhir

> **Bisakah nilai, state, ownership, concurrency, effect, protocol, dan biaya sebuah program dicatat sebagai satu entri ledger yang komposisinya mengikuti hukum sederhana?**

Hipotesisnya: bisa, untuk subset bahasa sistem yang terkontrol, jika correctness dipandang sebagai transisi state, ownership sebagai resource linear, effect sebagai grade, concurrency sebagai interaksi partial-order, dan biaya sebagai kuantitas dalam aljabar yang sama.

Paradigmanya bergeser dari *program sebagai fungsi* ke *program sebagai transformer resource yang terverifikasi*. Korespondensi di bawah adalah **analogi kerja, bukan teorema**. Ia baru layak disebut "Curry–Howard" bila konjektur di bagian 11 terbukti.

Sisi program	Sisi logika
Program	Bukti
Resource	Proposisi linear
Protocol	Bukti concurrent
Effect	Operasi aljabar
Biaya	Grade kuantitatif
Komposisi	Komposisi bukti

Referensi

Setiap referensi menautkan ke PDF penulis, arXiv, atau DOI penerbit.

- ZBeam — README, status MVP dan non-goals
- Krebbers dkk. The Essence of Higher-Order Concurrent Separation Logic, ESOP 2017
- Lecture Notes on Iris
- Jung dkk. Iris from the Ground Up, JFP 2018
- Mével, Jourdan, Pottier. Time Credits and Time Receipts in Iris, ESOP 2019
- de Vilhena, Pottier. A Separation Logic for Effect Handlers, POPL 2021
- Hinrichsen, Bengtson, Krebbers. Actris: Session-Type Based Reasoning in Separation Logic, POPL 2020
- Hoffmann, Hofmann. Amortized Resource Analysis with Polynomial Potential, ESOP 2010
- Hoffmann, Jost. Two Decades of Automatic Amortized Resource Analysis, MSCS 2022
- Das, Hoffmann, Pfenning. Work Analysis with Resource-Aware Session Types, LICS 2018
- Das dkk. Resource-Aware Session Types for Digital Contracts (Nomos), CSF 2021
- Wadler. Propositions as Sessions, ICFP 2012
- Plotkin, Pretnar. Handling Algebraic Effects, LMCS 2013
- Gordon. A Generic Approach to Flow-Sensitive Polymorphic Effects (effect quantales), ECOOP 2017
- Orchard, Wadler, Eades. Unifying Graded and Parameterised Monads, MSFP 2020

- Atkey. Syntax and Semantics of Quantitative Type Theory, LICS 2018
- Brady. Idris 2: Quantitative Type Theory in Practice, ECOOP 2021
- Orchard, Liepelt, Eades. Quantitative Program Reasoning with Graded Modal Types (Granule), ICFP 2019
- Rice. Classes of Recursively Enumerable Sets and Their Decision Problems, Trans. AMS 1953

Pembanding dan prior art (bagian 8):

- Jung dkk. RustBelt: Securing the Foundations of the Rust Programming Language, POPL 2018
- Verus
- Prusti
- Creusot
- CLR: Checker for Lifetimes and other Refinement types for Zig
- Zirakzigil
- zig-spec — spesifikasi Zig (informal dan mechanized)

Arah fundamental (bagian 11):

- Pruiksma, Pfenning. Back to Futures, JFP 2022
- Cave, Ferreira, Panangaden, Pientka. Fair Reactive Programming, POPL 2014
- Bahr, Graulund, Møgelberg. Diamonds are not forever: Liveness in Reactive Programming with Guarded Recursion, POPL 2021
- Congard, Munch-Maccagnoni, Douence. Linear Effects, Exceptions, and Resource Safety: a Curry-Howard Correspondence for Destructors, arXiv 2025
- Marshall, Wagner, Li, Gierczak, Ahmed. From Linearity to Borrowing, OOPSLA 2025
- Das, Pfenning. Rast: A Language for Resource-Aware Session Types, LMCS 2022
- uringscope: Portable, Low-Overhead Observability for io_uring, arXiv 2026
- withoutboats. Notes on io-uring
- rustfs-uring — cancel-safe io_uring read backend

Peta posisi (bagian 12):

- Fil-C — memory-safe C/C++
- InvisiCaps: The Fil-C Capability Model
- LWN. Fil-C: A memory-safe C implementation, Oktober 2025
- Fil-C ABI proposal — issue tracker Zig